

Multimodal Graph-RAG for Structured Financial Document Retrieval

Anubhav Maurya

Independent Researcher

github.com/W-Anubhav

manubhav619@gmail.com

Abstract—Retrieval-Augmented Generation (RAG) systems often struggle with enterprise documents containing dense tables and complex visual layouts. Traditional vector-based pipelines flatten document structure into text chunks, which can lead to incorrect numerical retrieval and hallucinated responses.

This paper presents a multimodal Graph-RAG architecture designed to preserve both semantic content and spatial document structure during ingestion. The proposed pipeline combines LayoutLMv3 and Table Transformer for layout extraction, stores narrative text in a Qdrant vector database, and maps structural relationships into a Neo4j property graph. A routing module dynamically selects the appropriate retrieval path based on query type.

Experiments conducted on the Nvidia Q3 FY25 CFO Commentary document demonstrate improved performance on tabular retrieval tasks compared to a standard vector-only RAG baseline, increasing retrieval accuracy from 40% to 90% on a small benchmark dataset. These results suggest that separating semantic and structural memory can improve reliability for document question-answering systems involving complex tables.

Index Terms—Retrieval-Augmented Generation, Graph Databases, Neo4j, Document AI, Computer Vision, Financial Document QA

I. INTRODUCTION

Large Language Models (LLMs) are increasingly used for enterprise document analysis and question answering. Retrieval-Augmented Generation (RAG) improves response reliability by injecting retrieved context into the model prompt. While effective for narrative text, conventional RAG pipelines often struggle with documents containing visually structured information such as tables, invoices, and financial reports.

Most vector-based retrieval systems process PDFs using linear text extraction. During this process, the spatial structure of tables is lost, causing row and column relationships to collapse into unstructured text chunks. As a result, language models frequently misinterpret numerical values or retrieve incorrect entries from tabular data.

This work proposes a multimodal Graph-RAG architecture that separates semantic retrieval from structural document representation. Instead of storing all information in a single vector database, the system uses two specialized storage layers:

- A vector database for semantic narrative retrieval
- A graph database for spatial and structural document relationships

A routing module dynamically selects the appropriate retrieval pipeline depending on the user query. The goal is to

improve the reliability of question answering over visually dense financial documents.

II. RELATED WORK

A. Vector-Based RAG Systems

Traditional RAG systems rely on dense vector embeddings to retrieve semantically similar text chunks. These approaches perform well for narrative retrieval tasks but do not preserve document layout or structural relationships.

B. Multimodal Document Understanding

Recent document understanding models such as LayoutLMv3 incorporate both textual and spatial information during document processing. Similarly, Table Transformer (TATR) improves table detection and structure recognition in scanned documents and PDFs.

C. Graph-Based Retrieval

Graph databases have recently been explored for representing relationships between entities and structured knowledge. In document processing pipelines, graph representations can preserve hierarchical document structure more effectively than flat text chunking approaches.

III. SYSTEM ARCHITECTURE

The proposed system consists of three primary stages:

- 1) Multimodal document ingestion
- 2) Dual-database storage
- 3) Query routing and retrieval

A. Architecture Overview

B. Multimodal Document Ingestion

Instead of relying on conventional PDF text extraction, each document page is converted into an image before processing. The ingestion pipeline uses two computer vision models:

- **LayoutLMv3**: Extracts textual content and normalized bounding box coordinates for document elements.
- **Table Transformer (TATR)**: Detects table regions and separates headers, rows, and cells.

This approach preserves spatial layout information that is typically lost during standard text extraction.

IV. DUAL-DATABASE STORAGE

After document parsing, the extracted information is divided into semantic and structural representations.

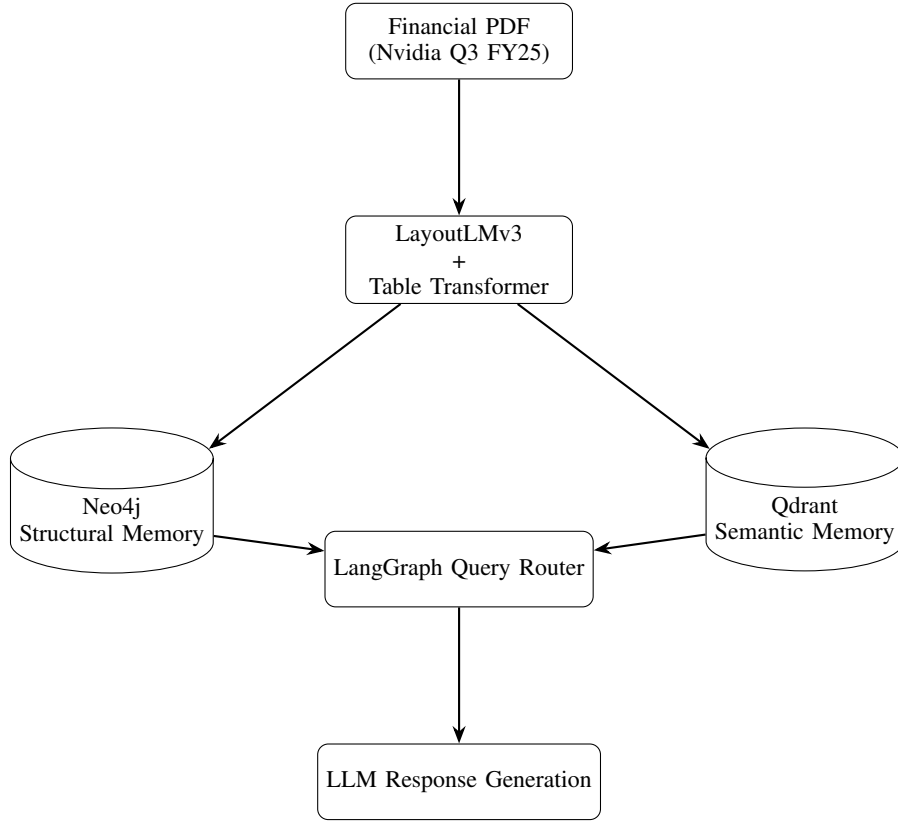


Fig. 1. Overview of the proposed multimodal Graph-RAG architecture. Structural document relationships are stored inside Neo4j, while narrative text is stored in Qdrant. A routing module dynamically selects the retrieval pipeline based on query type.

A. Semantic Storage Layer

Narrative text is split into 500-token chunks using a recursive text splitter and embedded into a Qdrant vector database. This layer supports semantic retrieval for descriptive or contextual queries.

B. Structural Storage Layer

Spatial relationships extracted from document layouts are stored inside a Neo4j property graph. The graph preserves hierarchical document structure using nodes such as:

Document → Page → Table → Cell

Bounding box coordinates are serialized before insertion into Neo4j to simplify graph storage and traversal.

C. Example Cypher Query

The following query illustrates how the system retrieves tabular values from the graph database:

```

MATCH (d:Document)-[:MENTIONS]->(c:Company)-[:
  PRODUCES]->(p:Product)
RETURN p.id, SUM(d.id) AS total_revenue
ORDER BY total_revenue ASC
LIMIT 1

```

V. QUERY ROUTING

The retrieval layer uses a LangGraph-based routing module to determine which database should handle a query.

- Narrative or contextual queries are routed to the vector database.
- Numerical or structurally dependent queries are routed to the graph database through generated Cypher queries.

A. Example Queries from Benchmark Dataset

The benchmark dataset was divided into narrative and structural query categories.

1) Narrative Query Examples:

- “What were the primary drivers for the record Data Center revenue this quarter?”
- “How did the Gaming segment perform, and what drove its results?”
- “What is the management’s general financial outlook for the next quarter?”

These queries primarily require semantic retrieval and contextual summarization.

2) Tabular Query Examples:

- “Exactly how much was spent on share repurchases versus cash dividends?”
- “What was the total operating expenses figure reported in millions?”

- “What is the exact Diluted Earnings Per Share (EPS) for this quarter?”

These queries require preservation of row-column relationships and exact numerical retrieval from financial tables.

VI. EXPERIMENTAL SETUP

A. Dataset

Evaluation was conducted using the Nvidia Q3 FY25 CFO Commentary document. A benchmark dataset containing 20 manually created evaluation queries was constructed using the source document.

The evaluation set consisted of:

- 10 narrative retrieval queries
- 10 tabular retrieval queries

Narrative questions focused on contextual understanding, while tabular questions required exact numerical retrieval from financial tables.

B. Evaluation Methodology

Each query was manually evaluated against the expected answer defined in the benchmark dataset.

A response was considered correct if:

- The retrieved numerical value exactly matched the source document for tabular queries.
- The generated answer captured the primary semantic meaning for narrative queries.

The proposed Graph-RAG architecture was compared against a standard vector-only RAG baseline implemented using LangChain and PyPDF-based text extraction.

VII. RESULTS

TABLE I
TABULAR RETRIEVAL PERFORMANCE

Architecture	Accuracy	Failure Rate
Vector-Only RAG	40%	60%
Proposed Graph-RAG	90%	10%

The vector-only baseline frequently failed on queries requiring precise table interpretation because row-column relationships were lost during text extraction.

The proposed Graph-RAG architecture improved retrieval accuracy on tabular queries by preserving spatial structure inside the graph database. The routing module successfully directed structural queries to the graph retrieval pipeline, allowing accurate extraction of numerical values from tables.

For example, the system successfully retrieved the following exact values from the Nvidia financial report:

- Operating expenses: \$4,287 million
- Share repurchases: \$11.0 billion
- Data Center revenue: \$30,771 million

VIII. LIMITATIONS

The proposed system introduces additional ingestion and retrieval overhead compared to conventional vector-only RAG systems.

The computer vision models also impose token and image processing limits on dense document pages. In addition, the current evaluation dataset is relatively small and limited to financial documents.

Some narrative benchmark questions also relied on information not explicitly present in the evaluated CFO commentary document, which may affect consistency during evaluation.

Future work should evaluate the architecture across larger document collections and additional enterprise domains.

IX. CONCLUSION

This work explores the limitations of vector-only RAG systems when applied to visually structured enterprise documents. By separating semantic retrieval from spatial document representation, the proposed architecture improves tabular question-answering performance on financial reports.

Although the current implementation introduces additional ingestion and retrieval overhead, the results demonstrate that graph-based structural memory can provide more reliable access to tabular information than conventional text chunking approaches.

Future work will focus on improving scalability, reducing retrieval latency, and expanding evaluation across more diverse datasets.

REFERENCES

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [2] Y. Xu et al., “LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking,” *ACM MM*, 2022.
- [3] M. Smock et al., “PubTables-1M: Towards Comprehensive Table Extraction From Unstructured Documents,” *CVPR*, 2022.
- [4] Neo4j Documentation. Available: <https://neo4j.com/docs/>
- [5] Qdrant Documentation. Available: <https://qdrant.tech/documentation/>
- [6] LangChain Documentation. Available: <https://python.langchain.com/>